

# **Introduction to Artificial Intelligence | 887810**

## **Final Project**

### **Submitted by:**

Avishay Rachmany - ID: 208151472

Ofir Aharoni - ID: 209523646

**Lecturer:** Mr. Akiva Bruno Malka

**Teaching Assistant:** Mr. Regev Yehezkel Imra

**Date:** 15/03/2026

# 1 Introduction

This final project examines three supervised learning tasks, each designed to highlight a different aspect of machine learning methodology. The goal of the project is not only to obtain strong empirical results, but also to demonstrate a clear understanding of model selection, evaluation protocols, hyperparameter tuning, optimization behavior, and generalization performance. Accordingly, the analysis throughout the report is based on quantitative evaluation and visual evidence, and the discussion is aligned with the outputs produced by the submitted code.

The report is organized into three main parts. Part I addresses a regression problem using the Auto MPG dataset, with emphasis on modeling assumptions, the bias-variance tradeoff, optimization behavior, and generalization across Linear Regression, Polynomial Regression, and K-Nearest Neighbors (KNN) models. Part II studies a multi-class classification task on the CIFAR-10 dataset, comparing the performance and limitations of Logistic Regression, Linear SVM, and KNN. Part III presents the implementation and analysis of a neural network classifier in PyTorch, focusing on architecture design, optimization choices, regularization, and validation-based model selection.

In addition, an optional bonus section extends the PyTorch framework to a regression setting on the Auto MPG dataset. This part examines how a neural network can be adapted from classification to regression, and compares its performance to the best classical regression model developed in Part I.

## 2 Part I: Regression Task - Fuel Efficiency Prediction

In this part of the project, we study a supervised regression problem using the Auto MPG dataset. The goal is to predict a vehicle's fuel efficiency, measured in miles per gallon (MPG), from structured vehicle characteristics. In accordance with the assignment requirements, the purpose of this task is not only to obtain strong predictive performance, but also to demonstrate a clear understanding of modeling assumptions, model complexity, optimization behavior, and generalization. For this reason, each modeling decision is supported by both quantitative evaluation and visual evidence.

To ensure a valid evaluation protocol, the dataset was divided into training, validation, and test sets using an approximate 60:20:20 split. Model selection and hyperparameter tuning were performed using the validation set only, while the test set was kept untouched until the final stage. After selecting the final model according to validation performance, it was retrained on the combined development data, namely the training and validation sets, and then evaluated on the held-out test set. This procedure prevents test leakage and allows the final test metrics to reflect genuine out-of-sample generalization.

## 2.1 Exploratory Data Analysis

Before fitting any regression model, we performed an exploratory analysis of the dataset. An initial inspection of missing values showed that *horsepower* was the only feature containing missing entries, with six missing observations. This indicated that a missing-value treatment step would be required during preprocessing.

We then examined the distribution of the target variable, MPG. The histogram shows that most observations are concentrated roughly between 15 and 30 MPG, with a moderate right tail toward higher fuel-efficiency values. This suggests that highly fuel-efficient vehicles do appear in the dataset, but are less common than cars in the moderate range.

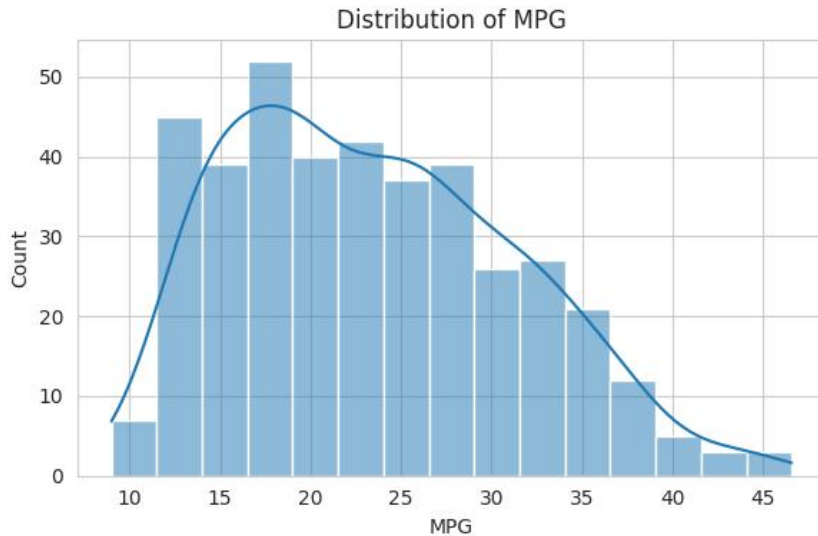


Figure 1: Distribution of MPG

Next, we investigated the relationships between the predictors and the target variable. The feature-target plots show clear negative relationships between MPG and *weight*, *horsepower*, and *displacement*, indicating that heavier, larger, and more powerful vehicles tend to be less fuel-efficient. In contrast, *model\_year* shows a clear positive trend, suggesting that newer cars in the dataset generally achieve better fuel efficiency. The boxplots for *cylinders* and *origin* reveal substantial group differences: cars with fewer cylinders tend to achieve higher MPG, and cars from Japan and Europe tend to be more fuel-efficient than cars from the USA. The relationship with *acceleration* appears weaker and noisier, but it still contains potentially useful information.

Finally, we examined correlations among the numerical variables. The correlation matrix confirms that *weight*, *displacement*, and *horsepower* are strongly negatively correlated with MPG, while *model\_year* is positively correlated with MPG. Several predictors are also strongly correlated with one another, especially among the engine-related and vehicle-size variables, indicating a meaningful degree of multicollinearity in the dataset.

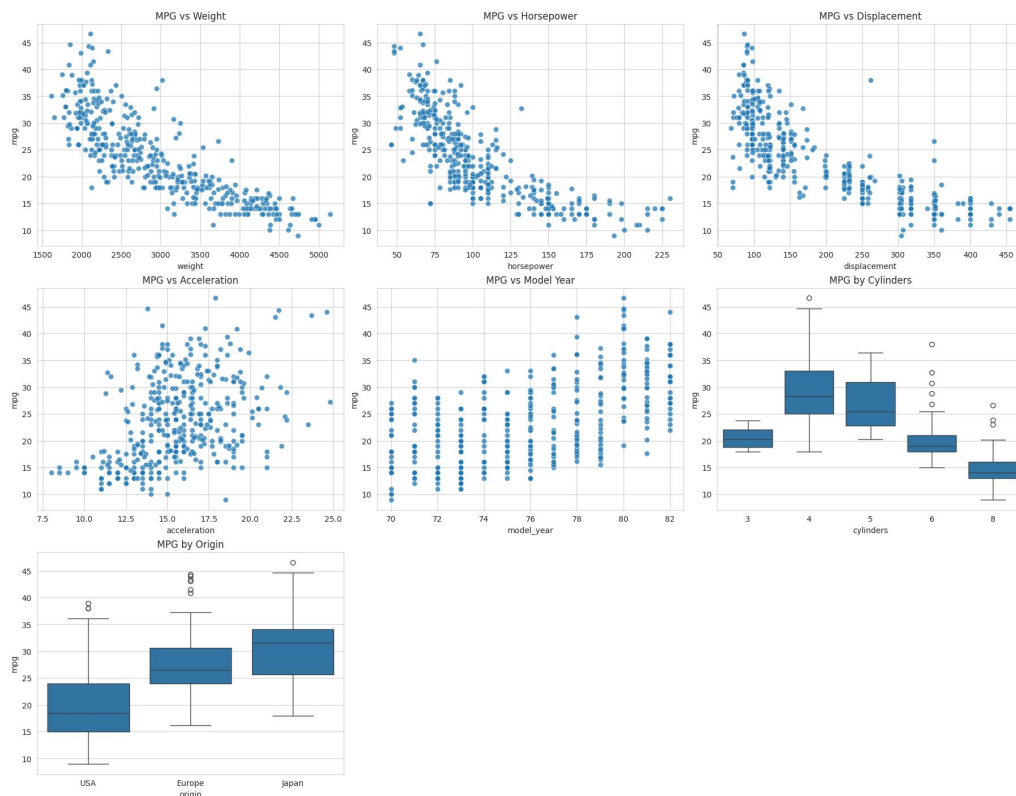


Figure 2: Feature-Target Relationships

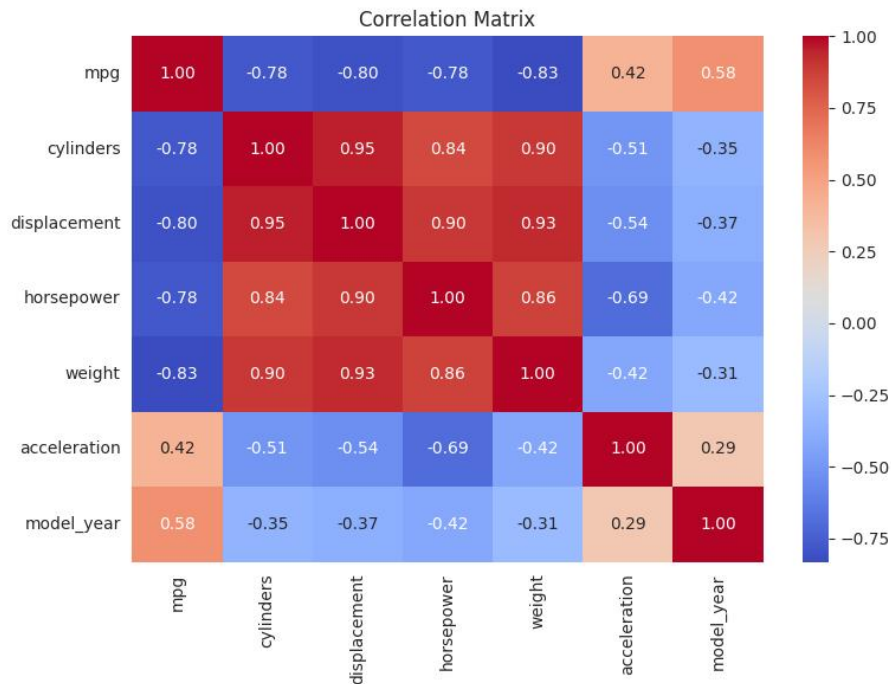


Figure 3: Feature Correlations Matrix

## 2.2 Dataset and Preprocessing

The target variable in this task is *mpg*, and the predictors used for modeling are *cylinders*, *displacement*, *horsepower*, *weight*, *acceleration*, *model\_year*, and *origin*, in accordance with the assignment specification. The textual *car\_name* field was excluded from the modeling pipeline, since this part focuses on structured vehicle attributes rather than text-based feature engineering.

The categorical variable *origin* was encoded using one-hot encoding with one reference category omitted. Missing values in *horsepower* were imputed using the median computed from the training data only, and the same value was then applied to the validation and test sets. This avoids data leakage and provides a robust treatment of missing values. Finally, all predictors were standardized using statistics computed from the training set only. Standardization was especially important for KNN, since distance-based methods are highly sensitive to feature scale, and it also ensured a consistent numerical representation across the linear and polynomial models.

## 2.3 Linear Regression Baseline

We began the modeling stage with Linear Regression, treating it as the simplest reasonable baseline for the problem. This model assumes an approximately linear relationship between the predictors and MPG and therefore provides an interpretable reference point for the more flexible models considered later.

On the validation set, the Linear Regression model achieved  $MSE = 15.305$ ,  $RMSE = 3.912$ ,  $MAE = 2.942$ , and  $R^2 = 0.794$ . These results indicate that even a simple linear model is able to capture a substantial portion of the structure in the data.

The prediction-versus-ground-truth plot shows a reasonable alignment with the diagonal reference line, but deviations become more pronounced at higher MPG values. In particular, the model tends to underpredict some of the more fuel-efficient vehicles. This suggests that the linear form captures the broad trend, but does not fully represent the complete structure of the relationship.

The residual plot provides additional evidence that the linearity assumption is only approximately valid. Although the residuals are centered around zero overall, their spread is not fully uniform, and a mild systematic pattern remains visible, especially in the higher predicted MPG range. This indicates that the linear model does not capture all patterns in the data equally well.

The coefficient plot is broadly consistent with the exploratory analysis. In particular, the large negative coefficient associated with *weight* and the positive coefficient associated with *model\_year* agree with the main trends observed in the data. At the same time, because several predictors are strongly correlated with one another, especially the engine-related variables, individual coefficient magnitudes should be interpreted with caution rather than in isolation.

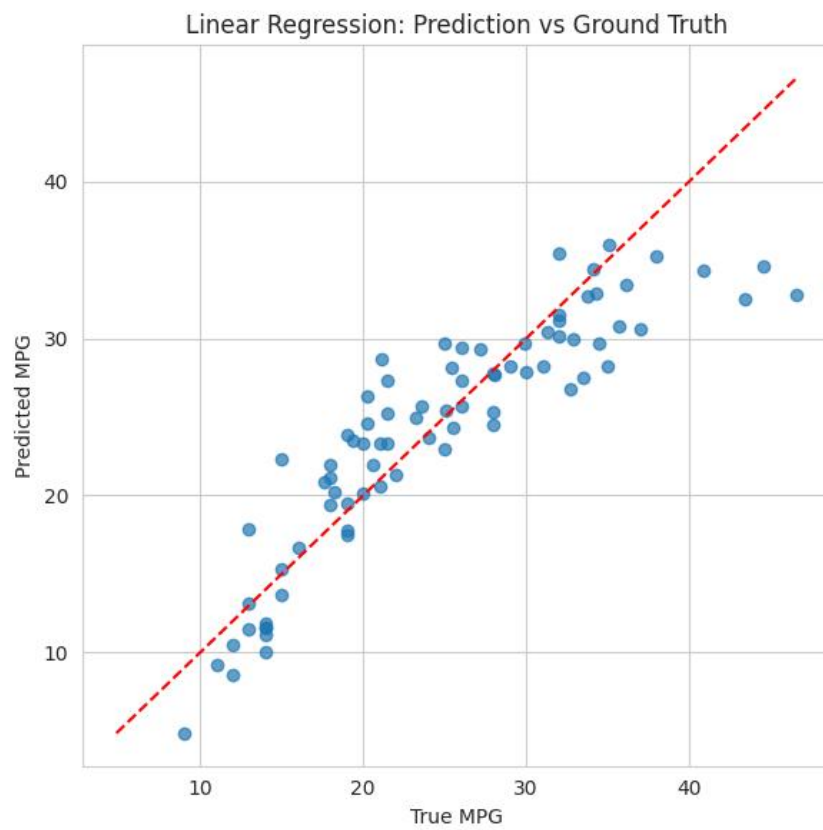


Figure 4: Linear Regression Prediction vs Ground Truth

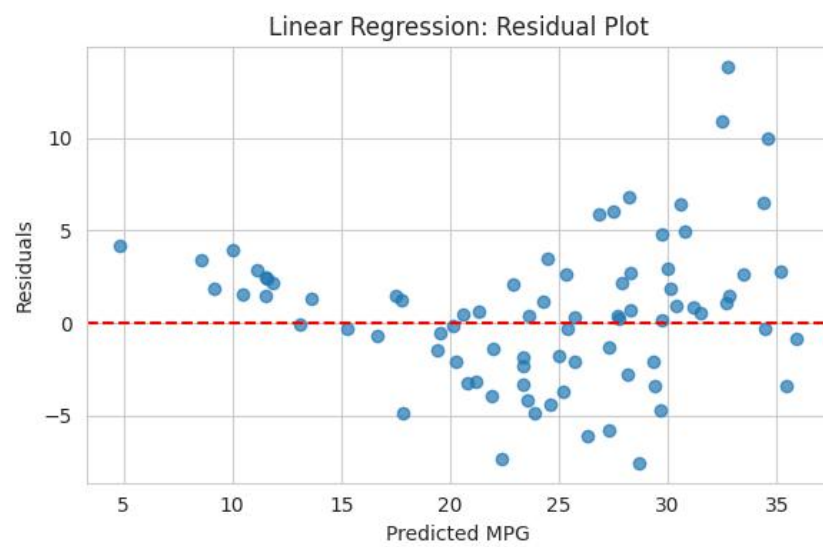


Figure 5: Linear Regression Residual Plot

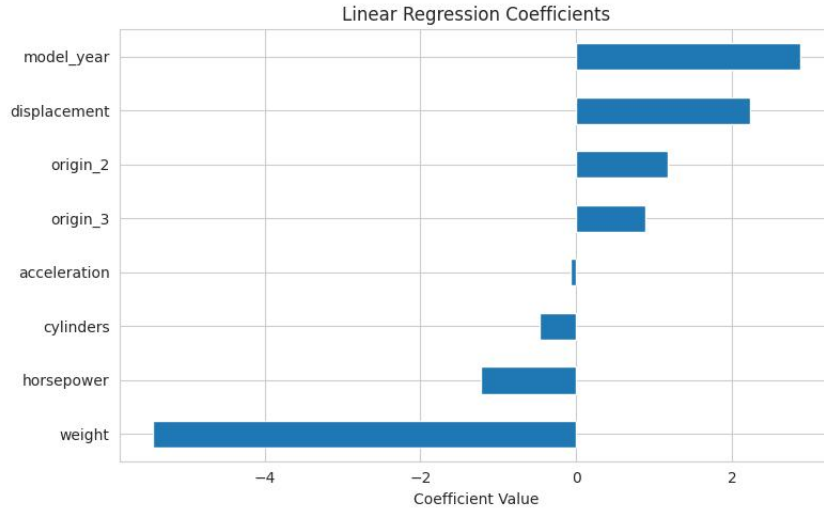


Figure 6: Linear Regression Coefficients

## 2.4 Polynomial Regression and Model Complexity

To increase model flexibility, we extended the linear model using polynomial features. Polynomial Regression models with degrees ranging from 1 to 7 were trained and compared using training and validation mean squared error.

The model-complexity curve clearly illustrates the bias-variance tradeoff. At low polynomial degrees, the model is too restrictive to capture the full structure of the data, which is consistent with underfitting. As the degree increases, the training error decreases sharply. However, beyond degree 2, the validation error increases substantially while the training error becomes extremely small, which is strong evidence of overfitting. Degree 2 therefore provides the best balance between flexibility and generalization in this experiment.

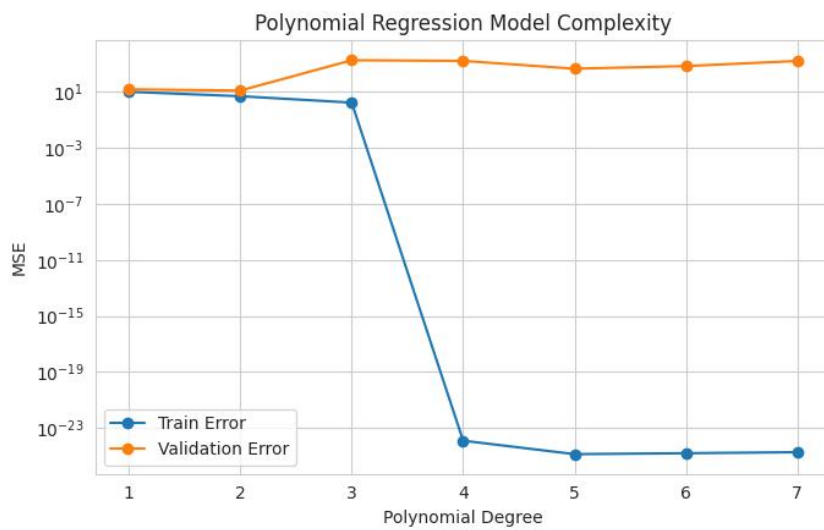


Figure 7: Polynomial Regression Model Complexity

Using degree 2, the final Polynomial Regression model achieved  $\text{MSE} = 12.466$ ,  $\text{RMSE}$

$= 3.531$ ,  $\text{MAE} = 2.497$ , and  $R^2 = 0.832$  on the validation set. This is a clear improvement over the linear baseline and indicates that moderate nonlinearity is beneficial for this task.

The prediction-versus-ground-truth plot for the degree-2 polynomial model provides a useful visual complement to the numerical results. Relative to the linear baseline, the predictions are more tightly concentrated around the diagonal, although noticeable dispersion still remains for some of the higher MPG observations.

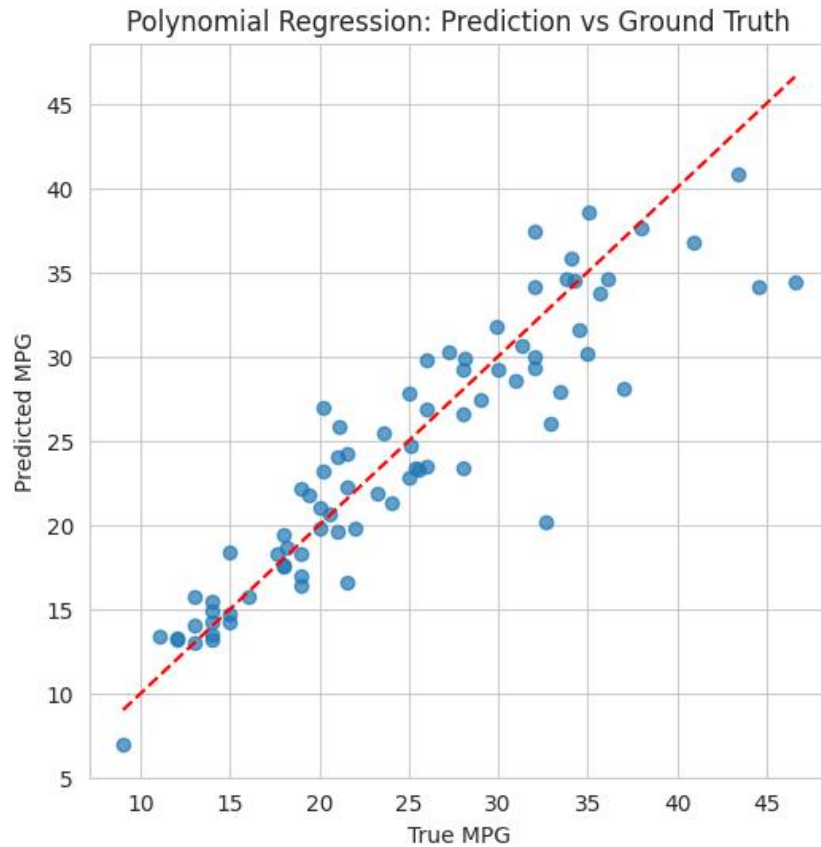


Figure 8: Polynomial Regression Prediction vs Ground Truth



## 2.5 K-Nearest Neighbors (KNN) Regression

We next considered a non-parametric regression model using K-Nearest Neighbors. In KNN regression, predictions are based on neighboring observations in feature space, so the value of  $k$  directly controls model smoothness and flexibility.

We evaluated values of  $k$  from 1 to 30 and selected the best value using validation performance. The train-and-validation error curve displays the expected bias-variance pattern. When  $k$  is very small, especially at  $k = 1$ , the model fits the training data extremely closely, which corresponds to high variance and a strong tendency to overfit. As  $k$  increases, the model becomes smoother and more stable, but excessive smoothing eventually increases bias and harms validation performance. The best validation performance was obtained at  $k = 4$ .

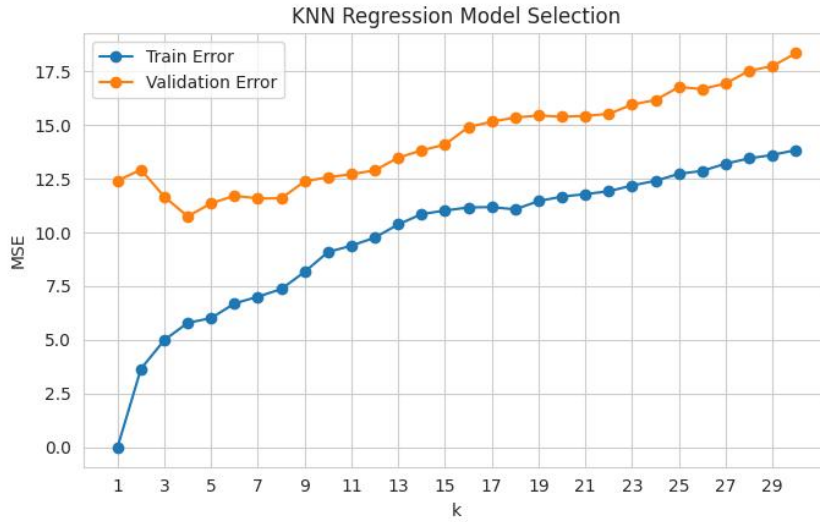


Figure 9: KNN Regression Model Selection

Using  $k = 4$ , the final KNN model achieved  $\text{MSE} = 10.751$ ,  $\text{RMSE} = 3.279$ ,  $\text{MAE} = 2.401$ , and  $R^2 = 0.855$  on the validation set, making it the strongest validation model among the three regression approaches.

The prediction-versus-ground-truth plot shows that KNN predictions generally lie closer to the diagonal than those of the linear baseline, which is consistent with the improved validation metrics. Some deviations remain, particularly at the upper end of the MPG range, but overall the fit is stronger.

KNN can outperform parametric models when the true relationship between predictors and target is nonlinear and local neighborhood structure is informative. However, it is less interpretable than linear models, and its inference-time cost is higher because each prediction depends directly on the stored training data.

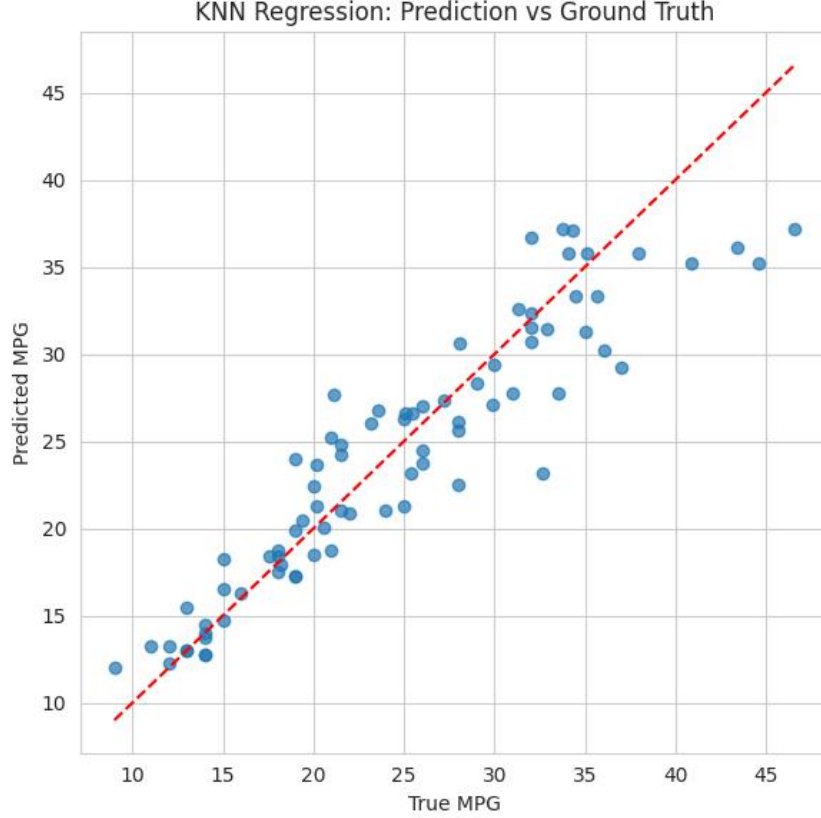


Figure 10: KNN Regression Prediction vs Ground Truth

## 2.6 Optimization Behavior

In addition to comparing regression models, we also analyzed the behavior of different optimization strategies on the same linear regression objective. Specifically, we compared Batch Gradient Descent and Mini-Batch Gradient Descent by tracking training and validation mean squared error across epochs.

The optimization curves show that Batch Gradient Descent decreases smoothly across epochs, which is expected because each update is computed from the entire training set. By contrast, Mini-Batch Gradient Descent converges much more rapidly in the early stages and reaches lower loss values more quickly in this experiment. Although mini-batch updates are computed from smaller subsets of the data, the epoch-level curves remain fairly stable here, while still reflecting the practical efficiency advantage of mini-batch optimization.

After 200 epochs, Batch Gradient Descent reached a final training loss of 11.467 and a final validation loss of 17.288, while Mini-Batch Gradient Descent reached a final training loss of 10.251 and a final validation loss of 15.481. Thus, in this setting, Mini-Batch Gradient Descent converged faster and achieved slightly better final validation behavior.

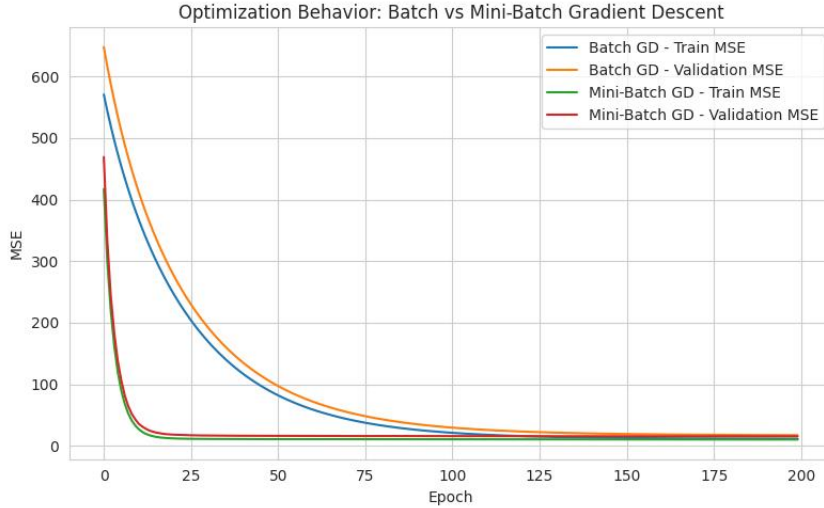


Figure 11: Optimization Behavior: Batch vs Mini-Batch Gradient Descent

## 2.7 Model Comparison and Discussion

We conclude the regression task by comparing the three candidate models: Linear Regression, Polynomial Regression, and KNN Regression.

In terms of predictive performance, KNN Regression achieved the best validation results, followed by Polynomial Regression and then Linear Regression. The polynomial model improved substantially over the linear baseline, confirming that the relationship between the predictors and MPG is not purely linear. At the same time, Linear Regression remained the most interpretable model, whereas KNN achieved the strongest predictive accuracy at the cost of lower interpretability and greater dependence on the training data during inference.

From a bias-variance perspective, Linear Regression exhibited the highest bias and the lowest flexibility. Polynomial Regression reduced bias by introducing controlled non-linearity, but higher polynomial degrees quickly led to severe overfitting. KNN, with an appropriately chosen value of  $k$ , provided the best balance on this dataset, capturing local structure without excessive smoothing.

From a practical perspective, we would select KNN Regression for deployment in this task because it achieved the best validation performance and also maintained strong performance on the held-out test set. Although it is less transparent than the linear model and more expensive at inference time, its predictive advantage is substantial enough to justify this choice.

After retraining the selected KNN model on the combined development data, we evaluated it on the held-out test set. The final model achieved  $MSE = 5.366$ ,  $RMSE = 2.316$ ,  $MAE = 1.916$ , and  $R^2 = 0.900$ . The final prediction-versus-ground-truth plot shows that the test predictions lie relatively close to the diagonal reference line, indicating strong generalization performance on unseen data. Some prediction error remains, especially for higher MPG values, but the overall agreement between true and predicted values is strong.

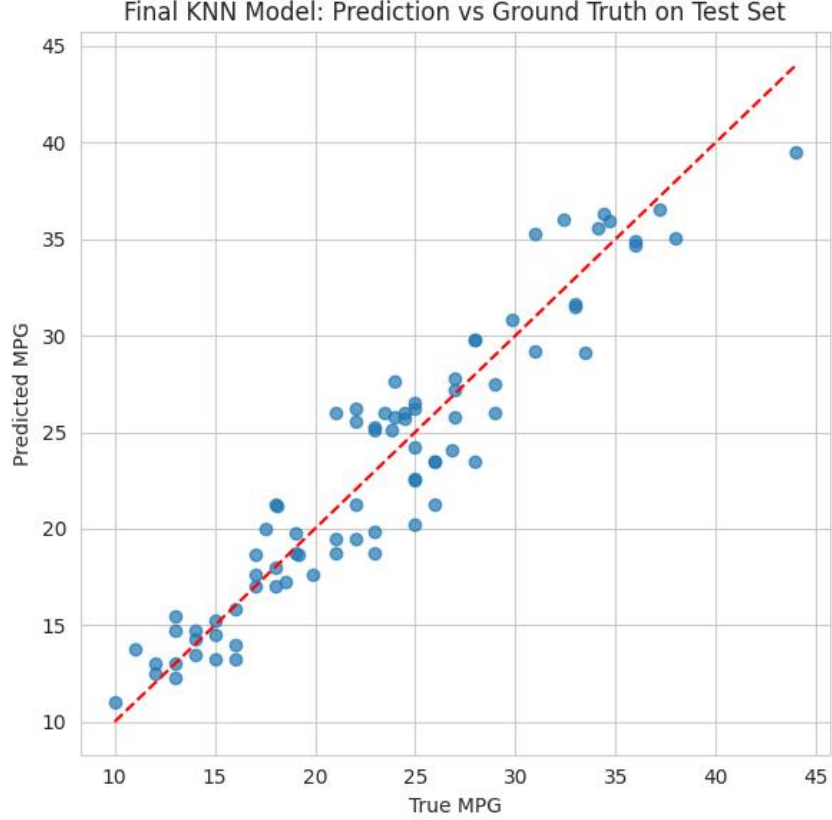


Figure 12: Final KNN Model Prediction vs Ground Truth on Test Set

## 3 Part II: Classical Classification on CIFAR-10

### 3.1 Experimental Setup and Preprocessing

In this part of the project, we studied a supervised multi-class classification problem using the CIFAR-10 dataset. The goal of this section was not only to compare predictive performance, but also to analyze how classical machine learning methods behave on high-dimensional image data and to understand their limitations. CIFAR-10 consists of  $32 \times 32$  RGB images from 10 semantic classes. After flattening, each image is represented by 3072 raw input features.

To ensure a fair comparison, all models were trained and evaluated using the same train-validation split. We started from the standard CIFAR-10 training set of 50,000 images and the standard test set of 10,000 images. The original training set was split into 40,000 training samples and 10,000 validation samples using a fixed random seed and stratified sampling. The test set was kept completely untouched throughout model comparison and hyperparameter tuning, and was used only once after the final model had been selected.

Since the classical models considered in this part operate on vector inputs rather than image tensors, each image was flattened into a one-dimensional feature vector of length 3072. We then applied standardization and Principal Component Analysis (PCA) within a single pipeline. Standardization was necessary because feature scale can strongly affect

both linear models and distance-based methods. PCA with 100 components was used to reduce dimensionality, improve computational tractability, and provide a more compact representation for the classifiers. Importantly, all preprocessing steps were fitted on the training data only and then applied consistently to the validation and test sets through the pipeline, thereby preventing data leakage.

## 3.2 Baseline Models

We implemented and evaluated the three classical models required in the assignment: Multiclass Logistic Regression (Softmax), Linear Support Vector Machine, and K-Nearest Neighbors. Each model was trained under the same data split and preprocessing protocol. This design makes the comparison methodologically fair and allows the observed performance differences to be attributed to the models themselves rather than to differences in data handling.

## 3.3 Hyperparameter Selection

For each model, at least one hyperparameter was explicitly selected using validation performance.

For Logistic Regression, we evaluated the regularization parameter  $C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$ . The highest validation accuracy obtained in this search was 0.3987, first attained at  $C = 0.1$ . We therefore selected  $C = 0.1$  as the final Logistic Regression configuration.

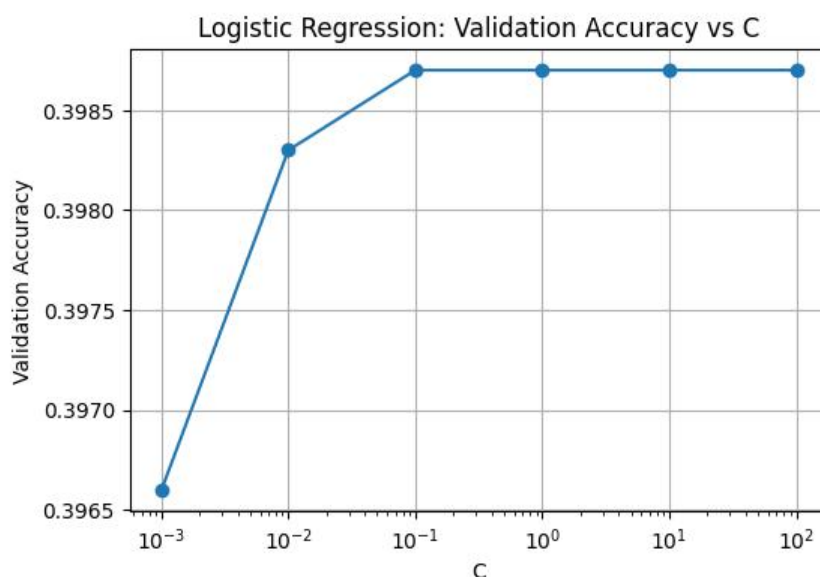


Figure 13: Logistic Regression Validation Accuracy vs C

The corresponding validation curve shows a clear improvement from very strong regularization to intermediate values of  $C$ , followed by a plateau. This indicates that once the regularization becomes sufficiently mild, additional increases in  $C$  do not provide a

meaningful validation gain. Therefore, choosing  $C = 0.1$  is both validation-consistent and methodologically reasonable, since it is the first value that attains the best result.

For Linear SVM, we evaluated the same range of regularization strengths,  $C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$ . The best validation accuracy was 0.3922, achieved at  $C = 0.01$ , which was selected as the final SVM hyperparameter.

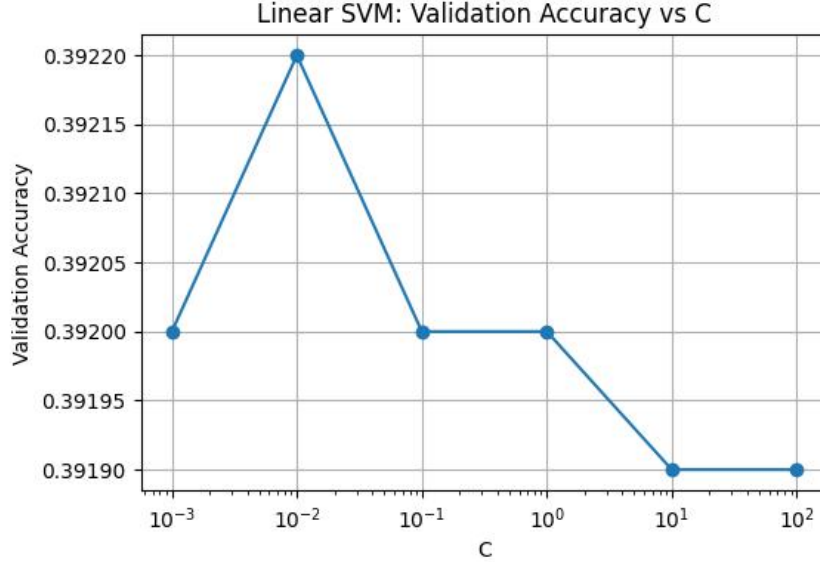


Figure 14: Linear SVM Validation Accuracy vs C

The validation curve shows a mild peak at  $C = 0.01$ , while the remaining values yield slightly weaker results. This suggests that the SVM is somewhat sensitive to regularization strength, although the overall variation remains relatively small. In other words, the model benefits from a moderate amount of regularization, but its performance does not change dramatically across the tested range.

For KNN, we evaluated  $k \in \{1, 3, 5, 7, 9, 11, 15\}$ . The best validation accuracy was 0.3817, achieved at  $k = 1$ . Although such a small value of  $k$  may increase variance, it provided the strongest validation performance among the tested candidates.

The validation curve shows that performance drops sharply when moving from  $k = 1$  to  $k = 3$ , then partially recovers for larger values of  $k$ , but never returns to the best result. This pattern is consistent with the bias-variance tradeoff in KNN: larger values of  $k$  smooth the decision rule and reduce variance, but in this dataset that smoothing appears to remove useful local information that helps classification.

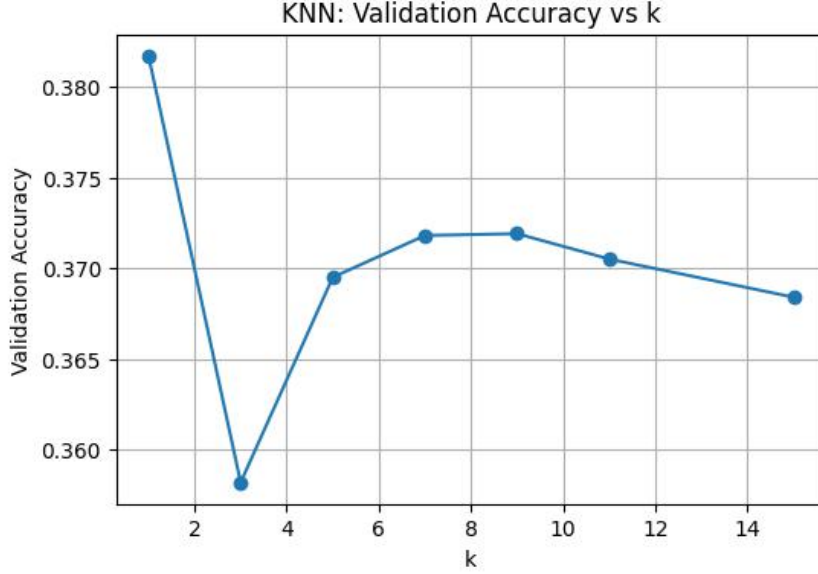


Figure 15: KNN Validation Accuracy vs k

### 3.4 Validation Evaluation and Model Comparison

The best validation results for each model family are summarized below:

| Model               | Best Hyperparameter | Best Validation Accuracy |
|---------------------|---------------------|--------------------------|
| Logistic Regression | $C = 0.1$           | 0.3987                   |
| Linear SVM          | $C = 0.01$          | 0.3922                   |
| KNN                 | $k = 1$             | 0.3817                   |

Table 1: Validation Evaluation and Model Comparison

Based on validation accuracy, Logistic Regression was selected as the final classical model. The gap between Logistic Regression and Linear SVM is relatively small, whereas KNN performs noticeably worse. This pattern is consistent with the difficulty of applying classical models to high-dimensional visual data. Even after dimensionality reduction, the feature space remains challenging, and the semantic structure of the classes is only partially captured by these methods.

In addition to reporting validation accuracy, we also examined the confusion matrix of each model on the validation set.

These confusion matrices provide a more detailed comparison than accuracy alone. Some classes, such as ship and frog, are recognized more reliably across models, while visually similar animal classes such as bird, cat, deer, and dog are often confused. KNN shows especially strong confusion between several animal classes, suggesting that Euclidean neighborhood structure in the PCA-reduced space is still insufficient for capturing the class structure of the data. Logistic Regression and Linear SVM perform slightly better, but their confusion patterns still indicate substantial limitations. In particular, the validation confusion matrices make it clear that moderate overall accuracy can hide



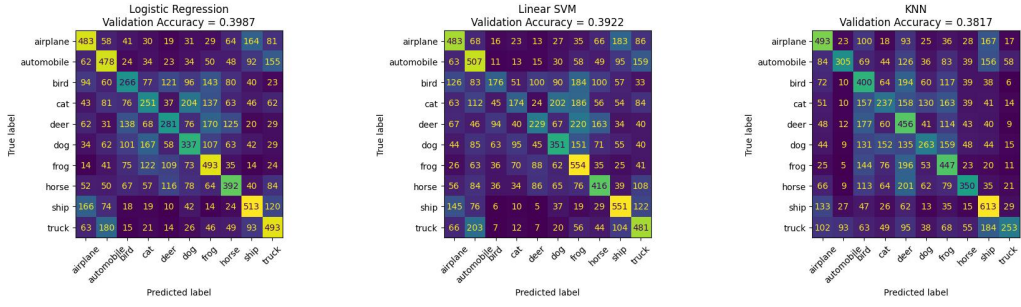


Figure 16: Confusion Matrices for Baseline Models

important structural weaknesses in class-wise recognition.

### 3.5 Effect of High Input Dimensionality and Limitations of Classical Models

This experiment highlights an important characteristic of CIFAR-10: although the images are relatively small, the raw input dimension is still high for classical methods once each image is flattened. Flattening destroys local spatial structure, such as edges, textures, and relative pixel arrangement, which are highly informative for image recognition. As a result, the models considered here cannot directly exploit the two-dimensional structure of the image.

Dimensionality reduction with PCA helps make the problem computationally feasible, but it does not fully solve the representation problem. The classifiers still operate on compressed vector representations rather than on spatially structured image features. This explains why all three classical models achieve only moderate validation performance and why confusion remains high among visually related classes. In this sense, the results in this part are not only a comparison of three models, but also an illustration of the broader limitations of classical machine learning on image data.

### 3.6 Final Model Selection and Test Evaluation

Following the required evaluation protocol, model selection was performed strictly using validation performance only. The test set was not used for model comparison or hyperparameter tuning.

After selecting Logistic Regression with  $C = 0.1$ , we retrained the model on the combined training and validation data and evaluated it once on the held-out test set. The final test accuracy was 0.4024.

The final test confusion matrix is consistent with the validation analysis.



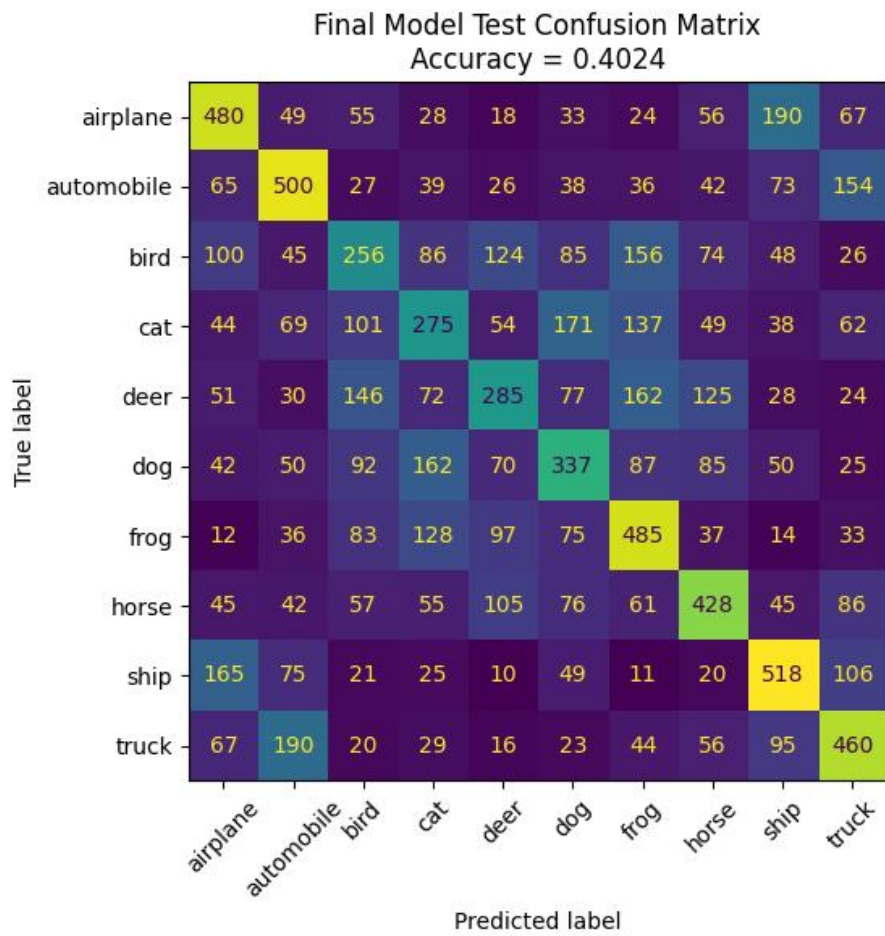


Figure 17: Final Model Test Confusion Matrix

### 3.7 Discussion

Among the three classical methods considered in this part, Logistic Regression achieved the strongest overall performance. Linear SVM performed slightly worse, and KNN was clearly the weakest model in our experiments. The relatively small gap between Logistic Regression and Linear SVM suggests that both linear decision boundaries capture some useful structure after PCA, but neither is capable of modeling image-specific spatial patterns.

Overall, the results show that classical machine learning methods can achieve reasonable, but limited, performance on CIFAR-10 after careful preprocessing and validation-based hyperparameter selection. This section therefore demonstrates both the usefulness and the limitations of classical approaches on high-dimensional image data, and it naturally motivates the transition to neural-network-based models in Part III.

## 4 Part III: Neural Network Classification with PyTorch

### 4.1 Objective and Experimental Setting

In this part of the project, we worked within the provided PyTorch notebook rather than rewriting it from scratch. The notebook already included the data loading pipeline, the training loop, and the evaluation procedure, and our task was to complete the missing implementation blocks and design a neural network classifier for CIFAR-10 while carefully justifying all architectural and optimization choices.

The goal of this section was not only to obtain strong classification accuracy, but also to follow a clear and reproducible model selection procedure. In particular, we compared several reasonable neural network alternatives, selected hyperparameters using validation performance only, and kept the test set untouched until the final evaluation. This directly follows the assignment requirements for the PyTorch part, which specify that the provided notebook should be completed through the existing TODO blocks, that hyperparameters must be chosen through code rather than guessing, and that the report should explain the final design decisions and the conclusions drawn from the comparisons.

### 4.2 Architecture Design

We began with a first comparison round across three candidate models representing two architecture families. The first model was a fully connected baseline with a single hidden layer of size 512 and ReLU activation, trained with SGD. The second was a more regularized fully connected network with two hidden layers of sizes 1024 and 512, Batch Normalization, ReLU activations, and Dropout with probability 0.3, trained with Adam and weight decay. The third was a simple convolutional neural network consisting of three convolutional blocks with channel dimensions 32, 64, and 128, where each block included

convolution, batch normalization, ReLU, and max pooling, followed by a classifier with a 256-unit fully connected layer, ReLU, Dropout, and a final 10-class output layer.

The motivation for this comparison was to test whether an architecture that explicitly exploits spatial structure provides a clear advantage on CIFAR-10. Since CIFAR-10 consists of small color images, convolutional layers are better suited to this task than purely fully connected layers because they can capture local spatial patterns and shared visual features more effectively.

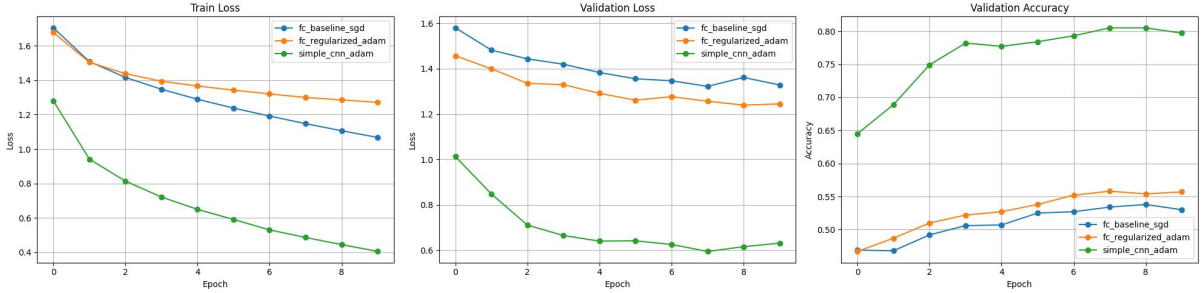


Figure 18: Visual Comparison of the First Round. Comparison between the fully connected baseline, the regularized fully connected model, and the CNN. The CNN achieves substantially better validation performance, motivating the focused search within the CNN family.

The first-round graph (Figure 18) clearly supports this architectural decision. In the training-loss panel, the CNN reduces training loss much more rapidly and much more substantially than either fully connected alternative. In the validation-loss panel, the CNN also reaches much lower values, indicating substantially better generalization. The validation-accuracy panel shows the same pattern even more clearly: while the two fully connected models remain in a relatively low accuracy range, the CNN rises to a much higher validation accuracy level and stays clearly separated from the fully connected baselines throughout training.

This conclusion is also reflected numerically. The fully connected baseline reached a best validation accuracy of 0.5380, the regularized fully connected model improved this to 0.5580, and the simple CNN reached 0.8050 in the first comparison round. Therefore, the main performance improvement came from moving to a convolutional architecture rather than merely changing the optimizer or adding regularization within a fully connected model.

Based on this result, we selected the CNN family for the second and more focused stage of model selection.

### 4.3 Hyperparameter Selection Procedure

After identifying the CNN as the strongest architecture family, we performed a focused comparison within that family. In this stage, we kept the overall CNN architecture fixed and compared several training configurations that differed in learning rate, dropout,

weight decay, and batch size. Model selection was based strictly on validation accuracy, while the test set was kept untouched until the final stage.

This procedure follows the assignment requirement that hyperparameters must not be selected by guessing, but rather through code that supports the final choice, such as a validation-based comparison or a small search. Our notebook therefore includes an explicit comparison between several CNN variants, together with the final selected configuration.

The five CNN configurations we compared were as follows:

1. Learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , batch size 64
2. Learning rate  $5 \times 10^{-4}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , batch size 64
3. Learning rate  $1 \times 10^{-3}$ , dropout 0.5, weight decay  $1 \times 10^{-4}$ , batch size 64
4. Learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $5 \times 10^{-4}$ , batch size 64
5. Learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , batch size 128

The focused comparison showed that the configuration with learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , and batch size 64 achieved the highest validation accuracy, equal to 0.8080. The remaining configurations achieved best validation accuracies of 0.8020, 0.7940, 0.7910, and 0.7820, respectively. Therefore, the first configuration was selected as the final model.

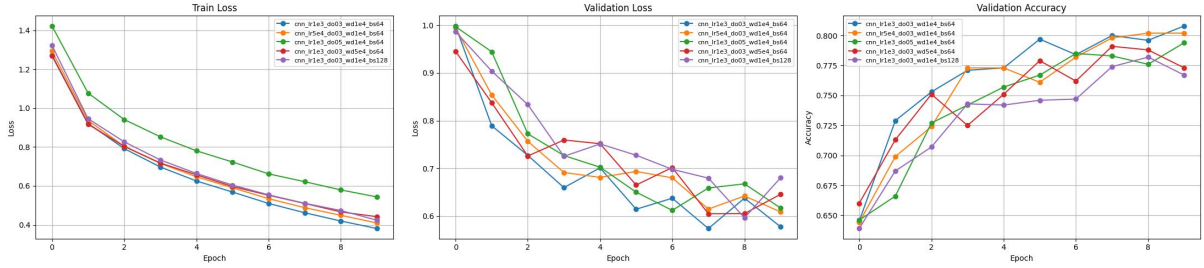


Figure 19: Visual Comparison of the Focused CNN Search. Comparison of CNN training configurations with different learning rates, dropout levels, weight decay values, and batch sizes. The configuration with learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , and batch size 64 achieved the best validation accuracy and was selected as the final model.

This graph (Figure 19) provides a detailed view of the behavior of the five CNN configurations during training. In the training-loss panel, all configurations improve steadily, which indicates that all of them learn meaningful representations. However, the model with dropout 0.5 consistently has the highest training loss, indicating slower optimization and stronger regularization. The selected configuration, with learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , and batch size 64, shows one of the cleanest and most effective reductions in training loss.

The validation-loss panel reveals a similar pattern. All configurations improve substantially over their initial values, but the selected configuration displays the most favorable overall validation-loss behavior. Several of the alternative configurations show somewhat larger fluctuations, which suggests less stable validation performance across epochs.

The validation-accuracy panel is the most important one for model selection, because validation accuracy was our predefined criterion. Here, the selected configuration reaches the highest peak validation accuracy, 0.8080, and remains among the strongest performers throughout training. The configuration with learning rate  $5 \times 10^{-4}$  is competitive but slightly weaker overall. Increasing dropout to 0.5 slows optimization and does not yield better peak validation accuracy. Increasing weight decay to  $5 \times 10^{-4}$  also does not improve the best validation accuracy in this run. Increasing the batch size to 128 leads to slightly weaker validation performance than batch size 64.

Overall, the focused CNN comparison supports two main conclusions. First, the CNN family is relatively robust, since several configurations achieve competitive performance. Second, the configuration with learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , and batch size 64 provides the best balance between optimization quality and validation performance, which justifies its selection as the final model.

## 4.4 Final Configuration and Test Performance

After completing model selection using validation accuracy only, we evaluated the selected model once on the held-out test set. The final chosen model was the CNN configuration with learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , and batch size 64.

This model achieved a validation accuracy of 0.8080 and a test accuracy of 0.7963. The gap between these two values is relatively small, which suggests that the selected model generalizes reasonably well and that the validation-based selection procedure was reliable. The final test loss was 0.6331.

This also follows the intended methodology of the assignment: the test set was not used for architecture comparison or hyperparameter tuning, and it was evaluated only after the final validation-based choice had already been made.

## 4.5 Main Conclusions

Several conclusions emerge from these experiments. First, architecture choice was the dominant factor in performance. The transition from fully connected models to a CNN produced a much larger improvement than later adjustments of individual hyperparameters. Second, within the CNN family, several configurations were competitive, but the configuration with learning rate  $1 \times 10^{-3}$ , dropout 0.3, weight decay  $1 \times 10^{-4}$ , and batch size 64 achieved the best overall validation performance. Third, stronger regularization did not automatically improve results. In our experiments, increasing dropout or weight decay did not lead to better validation accuracy and sometimes made optimization more

difficult.

Overall, this part demonstrates exactly the type of analysis required in the assignment: a justified architectural decision, validation-based hyperparameter selection, clear identification of the best configuration, and discussion of what was learned from the comparison process rather than reporting only a final score.

## 5 Bonus Task: Neural Network Regression with PyTorch

### 5.1 Objective and Adaptation of the Notebook

In this optional bonus task, we adapted the PyTorch notebook from the neural network classification part in order to solve the Auto MPG regression problem from Part I. Instead of rewriting the notebook from scratch, we preserved the general experimental workflow and modified it for a supervised regression setting. In particular, the classification-specific output, loss, and evaluation procedure were replaced with regression-oriented components, while the overall structure of model comparison, validation-based selection, and final held-out test evaluation was retained.

The goal of this section was not only to train a neural network that predicts the target variable *mpg*, but also to demonstrate a clear and reproducible model-selection process. This follows the assignment requirements for the bonus task, which emphasize adapting the PyTorch notebook from Part III, selecting models using validation performance, and comparing the final neural regressor to the best regression model from Part I.

### 5.2 Dataset and Preprocessing

We used the Auto MPG dataset from Part I, with *mpg* as the regression target. The input variables were the seven structured vehicle attributes: *cylinders*, *displacement*, *horsepower*, *weight*, *acceleration*, *model\_year*, and *origin*. In the notebook, missing values were handled by converting the non-numeric marker ‘?’ into missing values and removing incomplete rows. After this preprocessing step, the final usable dataset contained 392 observations.

The feature matrix was standardized using `StandardScaler`, fitted on the training split only and then applied to the validation and test splits. This step was important for stable neural-network optimization, since the input features are measured on different numerical scales. The data was divided into training, validation, and test sets, and all hyperparameter-tuning decisions were made using validation performance only. The test set remained completely untouched until the final evaluation stage.

### 5.3 Network Architecture and Regression Loss

To adapt the notebook from classification to regression, the most important architectural change was the output layer. In the classification notebook from Part III, the model produced a vector of class scores for the 10 CIFAR-10 classes. In the regression notebook, each candidate model produced a single scalar output corresponding to the predicted MPG value.

More broadly, the architecture itself also changed to match the structure of the data. In Part III, the best classification model was a convolutional neural network, which was appropriate for image inputs because convolution can exploit spatial structure. In the bonus task, the input consists of only seven standardized tabular features, so convolutional layers are not appropriate. For this reason, we compared a linear regressor and several multilayer perceptron (MLP) regressors based on fully connected layers.

The first round included three models: a linear baseline trained with SGD, a small MLP with one hidden layer of size 32 trained with Adam, and a regularized MLP with two hidden layers of sizes 64 and 32, ReLU activations, dropout, and Adam with weight decay. In the refined search, we further compared several fully connected MLP configurations with different hidden sizes, learning rates, dropout values, and weight decay settings.

For regression, we used mean squared error, implemented through `MSELoss`, as the training objective. This choice is natural for supervised regression because it directly penalizes prediction error and places greater weight on larger deviations. In addition to validation loss, we also tracked validation RMSE and validation MAE. Model selection was based on validation RMSE, since RMSE is expressed in the same units as the target variable and provides an interpretable measure of overall regression error. This is fully consistent with the requirement to adapt both the loss function and the evaluation procedure to the regression setting.

## 5.4 Initial Model Comparison

The first comparison round was designed to provide an initial picture of model behavior on the Auto MPG regression task. We compared three candidate models: `linear_sgd`, `mlp_small_adam`, and `mlp_regularized_adam`.

The results of this first round showed that the linear baseline was initially the strongest model. The `linear_sgd` configuration achieved the best validation RMSE in the first round, equal to 2.8658, whereas `mlp_small_adam` performed much worse with validation RMSE 13.6635, and `mlp_regularized_adam` improved substantially but still remained behind the linear baseline with validation RMSE 3.9224.

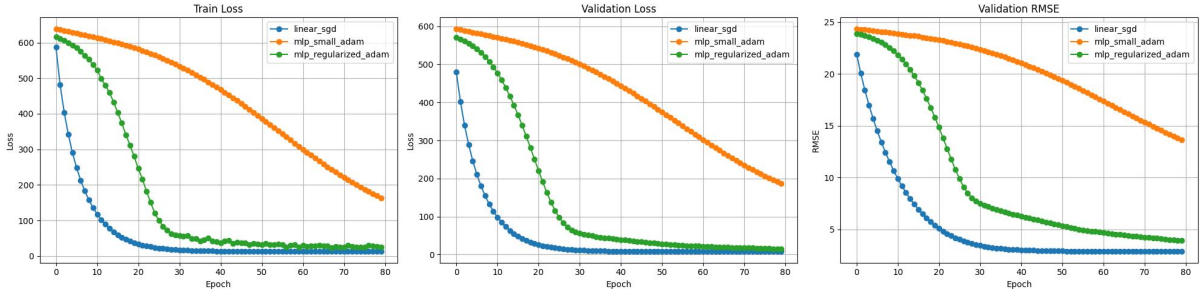


Figure 20: Visual Comparison of the First Round for Neural Network Regression.

This figure (Figure 20) is important because it immediately reveals two key insights. First, a simple linear model is a strong benchmark on this relatively small tabular dataset. Second, the weak performance of the initial neural networks should not yet be interpreted as evidence that neural models are inappropriate for the problem. Rather, the figure suggests that the first neural configurations were not yet sufficiently tuned in terms of architecture and optimization. This observation motivated a more focused second-stage search.

## 5.5 Refined Validation-Based Search

After the first comparison round, we performed a refined search around the most promising directions. In this stage, we compared several additional configurations that varied hidden layer sizes, optimizer settings, learning rate, dropout, and weight decay. The purpose of this search was to determine whether a better-tuned neural network could outperform the initial linear baseline.

The refined search included the following main candidates: two linear models trained with Adam at different learning rates, two MLPs with hidden sizes (64, 32) and different learning rates, one larger MLP with hidden sizes (128, 64), and one regularized version of that larger MLP with dropout and weight decay.

The refined search produced a clear and meaningful improvement. The best-performing configuration was `mlp_64_32_adam_1r5e3`, which achieved a validation RMSE of 2.7245 and validation MAE of 1.9793, and therefore became the final model selected for test evaluation. Other refined candidates were weaker, including `mlp_128_64_adam_1r1e3`



with validation RMSE 3.1816 and `mlp_128_64_adam_lr1e3_do01_wd1e4` with validation RMSE 3.3847, while the linear models trained with Adam performed much worse in this stage.

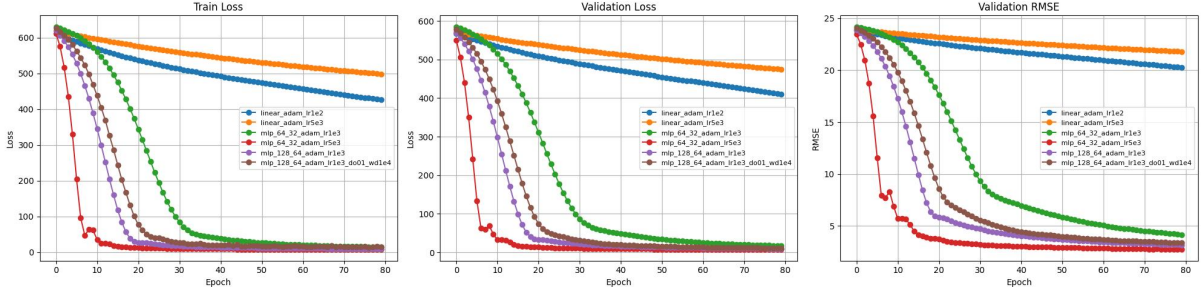


Figure 21: Visual Comparison of the Refined Search comparing different MLP and linear configurations.

The refined-search figure (Figure 21) provides strong support for this choice. The selected model shows the fastest and most substantial decrease in validation RMSE among all refined candidates. In contrast, the two linear models trained with Adam remain much weaker throughout training, while the larger or more regularized MLP variants improve but still do not reach the same validation quality. The graph therefore demonstrates that validation-based hyperparameter tuning was essential: although the initial comparison favored a linear model, the refined search showed that a properly tuned neural network could achieve better validation performance.

This stage also supports a broader methodological conclusion. The success of the refined search indicates that the early neural-network underperformance was not due to an inherent limitation of neural regression on Auto MPG, but rather to insufficient hyperparameter tuning in the first round.

## 5.6 Final Test Evaluation

After completing model selection using validation RMSE only, we evaluated the selected model once on the held-out test set. The final chosen network was `mlp_64_32_adam_lr5e3`, and it achieved Test RMSE = 2.6774 and Test MAE = 1.8612. The corresponding test loss was 7.1686.

These results are close to the validation performance used during model selection, which indicates that the selected network generalizes reasonably well and that the validation-based tuning procedure was reliable. The fact that the test RMSE is slightly lower than the validation RMSE is not problematic and may simply reflect a somewhat easier test split in this particular run.

A complementary qualitative perspective is provided by the prediction plot. Most of the points lie reasonably close to the diagonal reference line, meaning that predicted MPG values are generally close to the true target values. The overall alignment along the diagonal indicates that the network captured the main structure of the regression

relationship, while the remaining spread around the line reflects the residual prediction error that still remains.

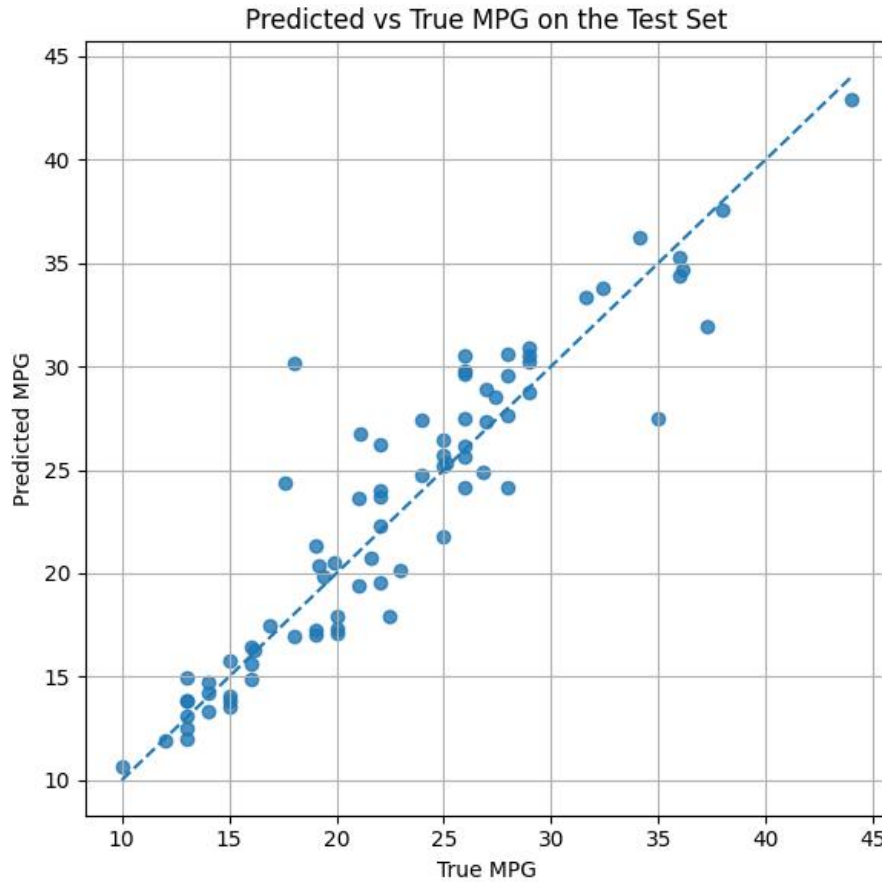


Figure 22: Predicted vs True MPG on the Test Set for the final MLP model.

Taken together, the numerical results and the prediction plot (Figure 22) support the conclusion that the final neural-network regressor achieves strong and stable performance on this dataset.

## 5.7 Comparison with the Best Regression Model from Part I

The best regression model from Part I was the K-Nearest Neighbors regressor with  $k = 4$ . After retraining on the full development set and evaluating on the held-out test set, it achieved  $\text{MSE} = 5.366$ ,  $\text{RMSE} = 2.316$ ,  $\text{MAE} = 1.916$ , and  $R^2 = 0.900$ .

In comparison, the neural-network regression model developed in the bonus task, namely `mlp_64_32_adam_1r5e3`, achieved  $\text{RMSE} = 2.6774$  and  $\text{MAE} = 1.8612$  on the held-out test set.

This comparison leads to a nuanced conclusion. On the one hand, the neural network achieved a slightly lower MAE, which indicates a somewhat smaller average absolute prediction error. On the other hand, the KNN model achieved a substantially lower RMSE, which means that it handled larger prediction errors more effectively and provided stronger overall predictive performance on this dataset. Since RMSE was the main

selection criterion in the bonus experiment and predictive performance was the central consideration in the final model choice in Part I, the KNN regressor remains the stronger overall model in the project.

At the same time, the bonus experiment remains highly informative. It shows that a carefully tuned neural network can achieve competitive regression performance on a small tabular dataset such as Auto MPG, even if it does not ultimately surpass the best classical method under the main evaluation criterion.

## 5.8 Main Conclusions

Several conclusions emerge from this bonus experiment.

First, replacing a classical regression model with a neural network does not automatically guarantee better performance. In the initial comparison round, the linear baseline remained the strongest model.

Second, a more focused hyperparameter search substantially improved the neural-network candidates and eventually identified a configuration that outperformed the earlier linear baseline on validation RMSE. This confirms that proper tuning is critical when evaluating neural models.

Third, the final selected network generalized well from validation to test. The similarity between validation and test performance, together with the visual structure of the prediction plot, supports the reliability of the model-selection process.

Fourth, the comparison with Part I shows that the neural network was competitive but not dominant. It achieved a slightly better MAE than the KNN regressor, but KNN remained better in RMSE and therefore remained the stronger overall model for this dataset.

Overall, this bonus task demonstrates that a PyTorch notebook originally built for classification can be successfully adapted to regression by modifying the output layer, the loss function, and the evaluation pipeline. It also reinforces one of the central lessons of the project: model flexibility alone is not enough, and strong performance depends on careful validation-based experimental design and rigorous comparison against meaningful baselines.

## 6 Conclusion

This project examined supervised learning from several complementary perspectives. In Part I, we compared classical regression models on the Auto MPG dataset and found that KNN regression provided the strongest overall predictive performance among the models considered. In Part II, we evaluated classical classification methods on CIFAR-10 and showed that, although careful preprocessing and validation-based tuning can yield reasonable results, classical models remain limited on high-dimensional image data. In Part III, the PyTorch experiments demonstrated that architecture choice was the dominant factor in performance, and that a convolutional neural network clearly outperformed fully connected alternatives on CIFAR-10. In the bonus task, we adapted the PyTorch framework to regression and found that a carefully tuned neural network achieved competitive performance on Auto MPG, although it did not surpass the best KNN regressor in terms of RMSE.

Overall, the project highlights a central lesson in machine learning: strong performance does not depend only on model flexibility, but also on appropriate representation, validation-based model selection, and careful comparison between alternative approaches. Across all parts of the project, the final conclusions were based on reproducible experiments, quantitative evaluation, and visual analysis.